

Base and Correction

The Multi-Core Sigmoid: local body detail the wings never feel, and the budget that keeps it honest

Vol-Fitter Technical Notes, No. 03 — lecture edition

Vol-Fitter

July 18, 2026

Abstract

Note 02 was about the wings: a smile's tails are a moment reading of the risk-neutral law, and one convex hyperbola pins them cleanly. This note fixes the wings as a solved problem and asks the complementary question — how to add detail in the *body* of a smile without disturbing the tails at all. Some smiles are not convex: ahead of a binary event, or in bimodally-positioned names, the curve develops a WW shape (a central trough, a shoulder on each side). We prove that no globally convex slice can carry such a shape, so a hyperbola is structurally disqualified, and we build the answer by *superposition*: a convex log-cosh *base* owns the wings and the overall skew, and R signed *corrections* add local humps or notches. The mathematical heart is a single fact — a centred second difference annihilates affine functions, and the log-cosh primitive is asymptotically affine, so each correction kernel and its first two derivatives vanish in both tails. The hats therefore reshape the body while leaving the base wing slopes *exactly* fixed: expressiveness and tail control are decoupled by construction. The model carries $6 + 4R$ parameters and a closed-form Durrleman diagnostic, and is fitted in three layered stages with an analytic Jacobian $2.36\times$ faster than finite differences. The second half of the note is the cost of flexibility. A flexible model over-fits: on a WW target two cores cut the miss from 153.05 to 2.55 vol bp, but on a liquid smile a third core lowers the in-sample error while *raising* the true-curve error and manufacturing butterfly arbitrage out in the sparse wing. So the cores slider is governed — an identifiability cap, an amplitude ridge, a hard cap at two, and a default-on put-wing regularizer that is zero on any clean slice. The cores slider is a scalpel, not a default.

Contents

1	Two jobs, one curve	3
2	The base owns the wings	5
3	A basis that vanishes in the tails	6
4	The full model: superposition	7
5	Static no-arbitrage	8
5.1	Where the arbitrage lives, and the put-wing regularizer	9
6	Calibration	10
7	Expressiveness on a budget	12
8	Worked example: a WW smile	14
9	What is original, and limitations	15
A	Hyperparameter atlas	16
B	Performance notes	17
C	Traceability	17
D	Reference implementation	18

1 Two jobs, one curve

A volatility smile does two jobs, and Note 02 did one of them. Far from the money the smile reports the *tails* of the risk-neutral distribution, and there one convex hyperbola — raw SVI — is enough: its two straight wings are a moment reading, capped by Lee’s bound. Near the money the smile reports the *body*, the local shape around the forward, and there a hyperbola runs out of room. Some smiles are simply not convex.

Proposition 1 (A convex slice cannot make a WW). *If total variance $w(k)$ is convex on an interval, it has no interior strict local maximum there, hence no shoulder flanked by two troughs. A WW smile — a central trough with a shoulder (interior local maximum) on each side — can therefore not be represented by any globally convex slice, in particular by raw SVI, whose $w'' > 0$ everywhere.*

Proof. A convex function on an interval has non-decreasing one-sided derivatives; an interior strict local maximum would force the derivative to fall from positive to negative, contradicting monotonicity. Raw SVI has $w'' = b s^2 / r^3 > 0$ (Note 02), so it is strictly convex and admits a single minimum only. $\square$

Figure 1 shows the situation. The convex base misses the two shoulders by 153.05 vol bp, and — the point of the whole note — the residual it leaves is *structured*: two localized shoulder features and a wing compromise, not white noise. A structured residual invites a structured correction. The design question is what that correction should be, and it has a sharp constraint: it must reshape the body without touching the wings, because the wings are the solved problem.

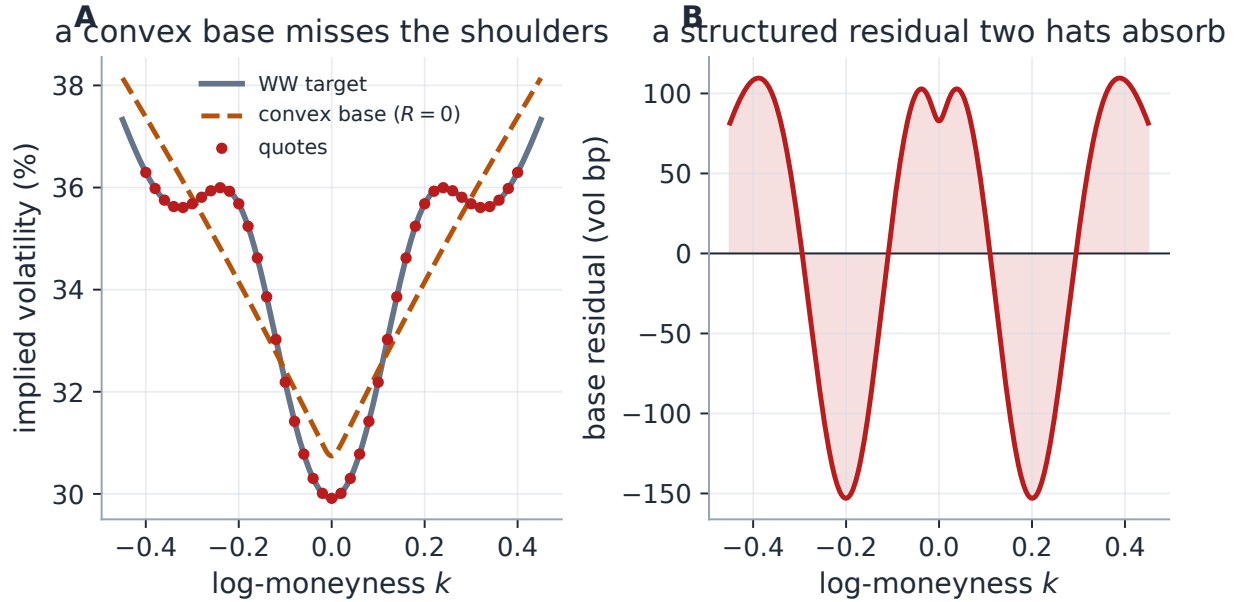


Figure 1: Why one convex base is not enough. Panel A: a WW target (trough plus two shoulders); the convex base ($R = 0$) tracks the overall V but misses both shoulders by 153.05 vol bp. Panel B: the base residual is structured — two shoulder features and a wing compromise — exactly the kind of thing a few *local* corrections can absorb.

Conventions and clocks. Throughout, $k = \log(K/F)$ is log-moneyness and $v = \sigma_{\text{imp}}^2$ is *annualized implied variance* — the quantity this model works in — while $w = \tau v$ is total variance. Here $\tau = \tau_T$ is the production *variance clock*, calendar time dilated by scheduled events (Notes 00, 11), equal to the calendar year fraction t only when the event clock is off; production passes τ to the calibrator, and wherever a bare t plays a time role below, read τ . The z -scale reference σ_{ref} is the *quoted* volatility nearest the money (with a median-of-quotes fallback when that quote is degenerate); it is fixed before the fit and never optimized. A *volatility basis point* (vol bp) is 10^{-4} in volatility. One differentiation convention: a prime is the derivative of a one-variable function (Φ' , B' , v'), a subscripted or explicit ∂ is a partial.

Symbol	Meaning	First used
k, z	log-moneyness; dimensionless log-strike $k/(\sigma_{\text{ref}}\sqrt{t})$	(1)
$v(z)$	annualized implied variance (the modelled quantity)	(4)
w, τ, t	total variance τv ; variance clock; calendar fraction	§1
σ_{ref}	reference vol fixing the z -scale	(1)
Φ_κ	log-cosh convexity primitive	(2)
V_0, S_0, K_0, z_0	base level, slope, convexity, centre (at z_0)	(4)
κ_P, κ_C	asymmetric put/call wing steepnesses	(4)
$B_{c,h,\kappa}$	zero-wing hat: centre c , half-width h , steepness κ	(6)
α_r, R	signed hat amplitude; number of cores	(7)
$g(k)$	Durrleman density factor (butterfly diagnostic)	(9)
β_P, β_C	k -space total-variance wing slopes	(8)

Table 1: Notation ledger. Nothing is reused with a second meaning; v is always variance, σ always a volatility.

Invariant

1. Adding cores never moves the asymptotic wings: every hat and its first two derivatives vanish in both tails (lemma 1), so the base owns the wing slopes regardless of R .
2. Expressiveness is budgeted: cores are capped by the quote count (eq. (11)) and the surfaced slider is clamped to two.
3. The model reports its own butterfly diagnostic g per fit; it is *not* arbitrage-free by construction, and the put-wing hinge rows are exactly zero on any clean slice — penalty off is byte-identical, penalty on over a clean slice unchanged to solver tolerance.
4. The parameters are a means, not the object: hats are non-unique by design, and only the fitted *curve* is contractual (section 6).

Heuristic

The correction must be *local*: it changes a bounded region of the body but must not alter the asymptotic wings, which liquid risk-reversals and Lee’s law already pin. So the kernel must vanish — in value, slope, *and* curvature — in both tails. “Zero-wing” does not uniquely select a kernel (a Gaussian and its derivatives also vanish), but the centred second difference of the same

log-cosh primitive the base uses is the convenient choice: model and Jacobian then share one set of analytic jets, the kernel is unit-height at its centre so its amplitude reads as a variance displacement, and its plateau width and shoulder steepness are separate dials (h, κ) where a Gaussian has only one.

2 The base owns the wings

Work in the dimensionless log-strike

$$z = \frac{k}{\sigma_{\text{ref}} \sqrt{t}}, \quad (1)$$

so z measures moneyness in standard deviations and the model is roughly scale-free across expiries. The convexity primitive is the *log-cosh*

$$\Phi_{\kappa}(u) = \frac{4}{\kappa^2} \log \cosh\left(\frac{\kappa u}{2}\right), \quad \Phi'_{\kappa}(u) = \frac{2}{\kappa} \tanh\left(\frac{\kappa u}{2}\right), \quad \Phi''_{\kappa}(u) = \text{sech}^2\left(\frac{\kappa u}{2}\right). \quad (2)$$

Φ_{κ} is a smooth, convex, even softened absolute value. Two facts about it carry the whole note. Near the origin $\Phi_{\kappa}(u) \approx u^2/2$ (a rounded vertex). Far out it is *asymptotically affine*,

$$\Phi_{\kappa}(u) = \frac{2}{\kappa} |u| - \frac{4}{\kappa^2} \log 2 + o(1), \quad (3)$$

a straight wing of slope $2/\kappa$ (fig. 2A). For numerical safety the log-cosh switches to its exact asymptote $|x| - \log 2$ beyond $|x| = 50$ to avoid cosh overflow — clipping x instead would freeze Φ constant and destroy the wing slopes.

The base slice and its z -derivatives are

$$\begin{aligned} v_{\text{base}}(z) &= V_0 + S_0(z - z_0) + K_0 \Phi_{\kappa_i}(z - z_0), \\ v'_{\text{base}}(z) &= S_0 + K_0 \Phi'_{\kappa_i}(z - z_0), \quad v''_{\text{base}}(z) = K_0 \Phi''_{\kappa_i}(z - z_0), \end{aligned} \quad (4)$$

with the steepness switching $\kappa_i = \kappa_P$ for $z < z_0$ and κ_C for $z \geq z_0$. The six parameters are the variance level V_0 at the fitted centre z_0 , the slope S_0 there, the convexity amplitude $K_0 \geq 0$, the centre z_0 , and the asymmetric put/call wing steepnesses κ_P, κ_C . Be precise about what V_0, S_0 are *not*: z_0 is fitted and generally $\neq 0$, so they are not the ATM level and skew, and the corrections of the next section can move the full model's ATM handles besides — the displayed ATM level, skew and curvature always come from the full curve. Because Φ and Φ' both vanish at the origin, the slice is C^2 across z_0 even though the steepness switches there: the asymmetry is in the *rate* at which each wing straightens, not a kink. By (3) the asymptotic wing slopes are

$$v'_{\text{base}}(z) \rightarrow S_0 \mp \frac{2K_0}{\kappa_{P,C}} \quad (z \rightarrow \mp\infty), \quad (5)$$

a left slope $S_0 - 2K_0/\kappa_P$ and a right slope $S_0 + 2K_0/\kappa_C$. These six numbers set the wings, and — the design goal — the corrections must leave them untouched.

3 A basis that vanishes in the tails

The correction kernel is the normalized centred second difference of the same primitive,

$$B_{c,h,\kappa}(z) = \frac{\Phi_\kappa(u-h) - 2\Phi_\kappa(u) + \Phi_\kappa(u+h)}{2\Phi_\kappa(h)}, \quad u = z - c, \quad (6)$$

centred at c , of half-width h and steepness κ , normalized so $B_{c,h,\kappa}(c) = 1$. Its derivatives are the same second difference of Φ' and Φ'' over the same normalizer. Now the one fact everything rests on.

Lemma 1 (Zero wings). $B_{c,h,\kappa}(z) \rightarrow 0$, $B'_{c,h,\kappa}(z) \rightarrow 0$ and $B''_{c,h,\kappa}(z) \rightarrow 0$ as $z \rightarrow \pm\infty$.

Proof. A centred second difference annihilates affine functions: for any $L(u) = pu + q$, $L(u-h) - 2L(u) + L(u+h) = 0$ identically. By (3) Φ is affine plus $o(1)$ in each tail, so its second difference tends to 0 there. The same argument applies to Φ' (which tends to the constants $\pm 2/\kappa$, affine of zero slope) and to Φ'' (which decays), so B , B' and B'' all vanish in both tails. $\square$

Lemma 1 is the whole design. Adding αB to the base changes the body but leaves the wing slopes (5) *exactly* unchanged. The amplitude α is in variance units and, since $B(c) = 1$, is approximately the variance displacement at the centre; a positive α raises a shoulder, a negative one digs a notch. The centre curvature $B''(c) = (\text{sech}^2(\kappa h/2) - 1)/\Phi_\kappa(h) < 0$, so a positive hat is locally concave (a hump), as a shoulder should be. Figure 2 draws both halves of the mechanism.

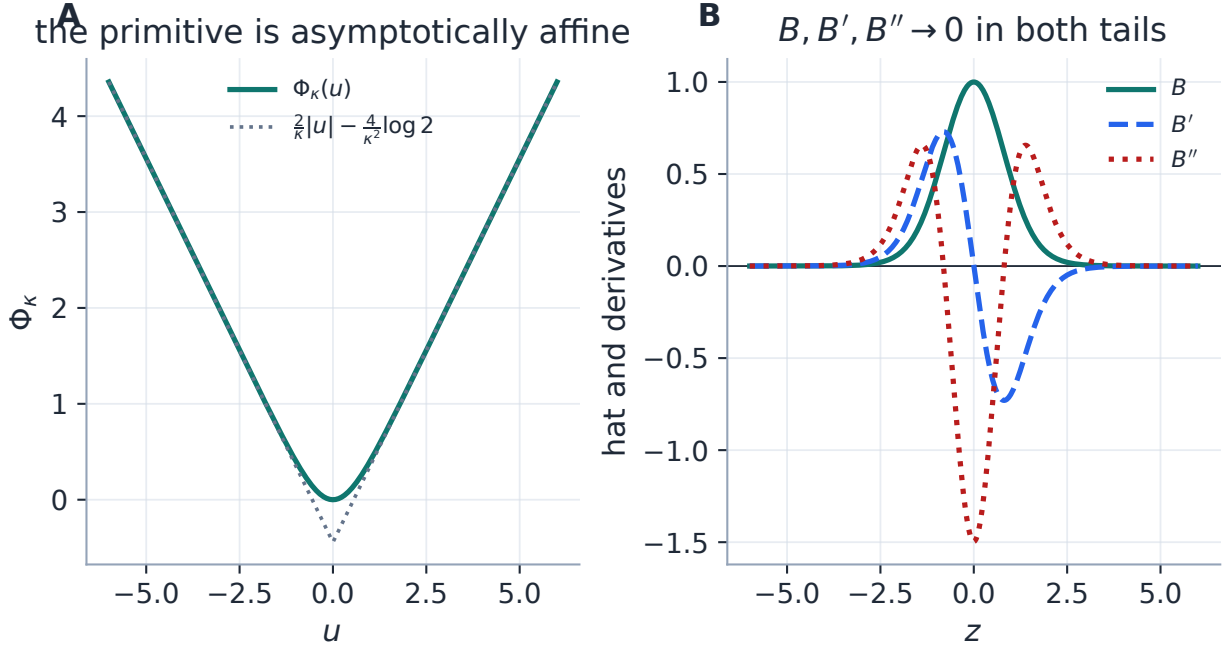


Figure 2: The mechanism, in two facts. Panel A: the log-cosh primitive is asymptotically affine — it merges with the straight line $\frac{2}{\kappa}|u| - \frac{4}{\kappa^2}\log 2$ in both tails, differing only at the rounded vertex. Panel B: the centred second difference of an asymptotically-affine function is a localized bump whose value, slope *and* curvature all decay to zero. A correction built this way is silent in the wings to all three orders.

Exercise 1. Verify the annihilation directly: for $L(u) = pu + q$, expand $L(u - h) - 2L(u) + L(u + h)$ and confirm it is identically zero. Conclude that *any* asymptotically-affine primitive — not just the log-cosh — yields a zero-wing kernel by the same second difference, and that the log-cosh is a convenience (shared jets, unit height, two dials), not a necessity.

4 The full model: superposition

The Multi-Core Sigmoid (MCS) slice is the base plus R signed corrections,

$$v_R(z) = v_{\text{base}}(z) + \sum_{r=1}^R \alpha_r B_{c_r, h_r, \kappa_r}(z), \quad w(k) = t v_R(z), \quad (7)$$

with $6 + 4R$ parameters. Figure 3 shows the decomposition on the WW fit: the convex base owns the V, and two signed hats add precisely the two shoulders, each localized. By lemma 1 every member of this family has the *same asymptotic wings as its base*, whatever R is — fig. 4 makes that visible, overlaying the base and the base-plus-hats: the bodies differ, the wings coincide, and the difference $\sum_r \alpha_r B_r$ decays to zero out of the money. Expressiveness in the body and control of the tails are, by construction, two separate concerns.

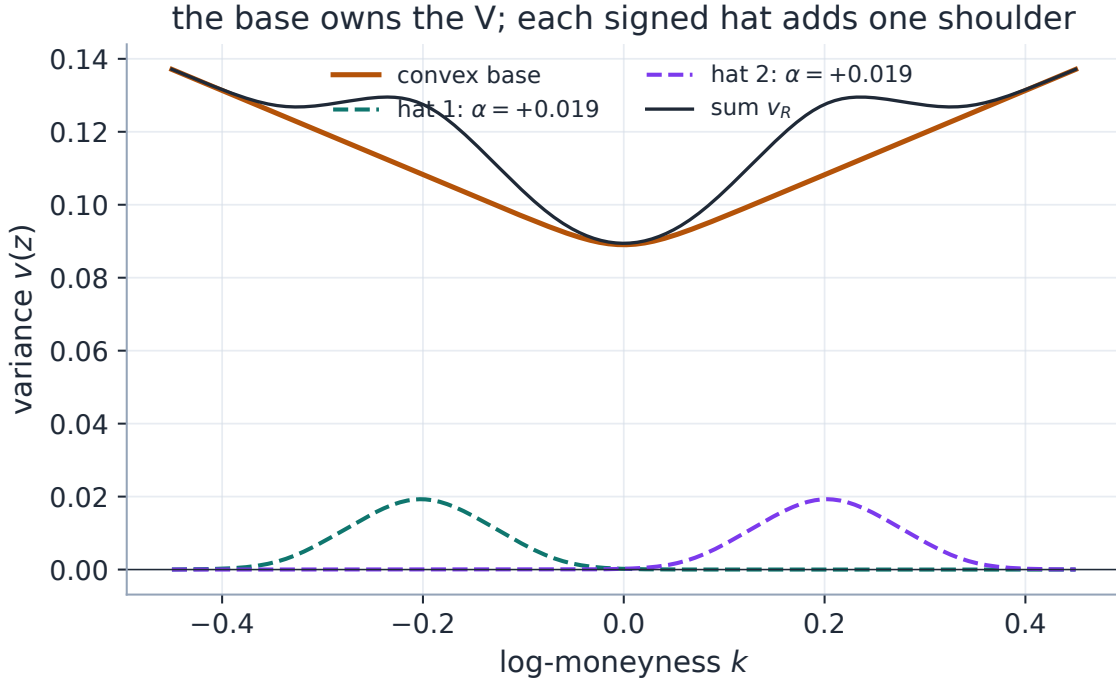


Figure 3: Superposition. The convex base carries the overall V; each signed hat is a localized bump that adds one shoulder; their sum is the fitted slice. The hats live where the residual of fig. 1B lived.

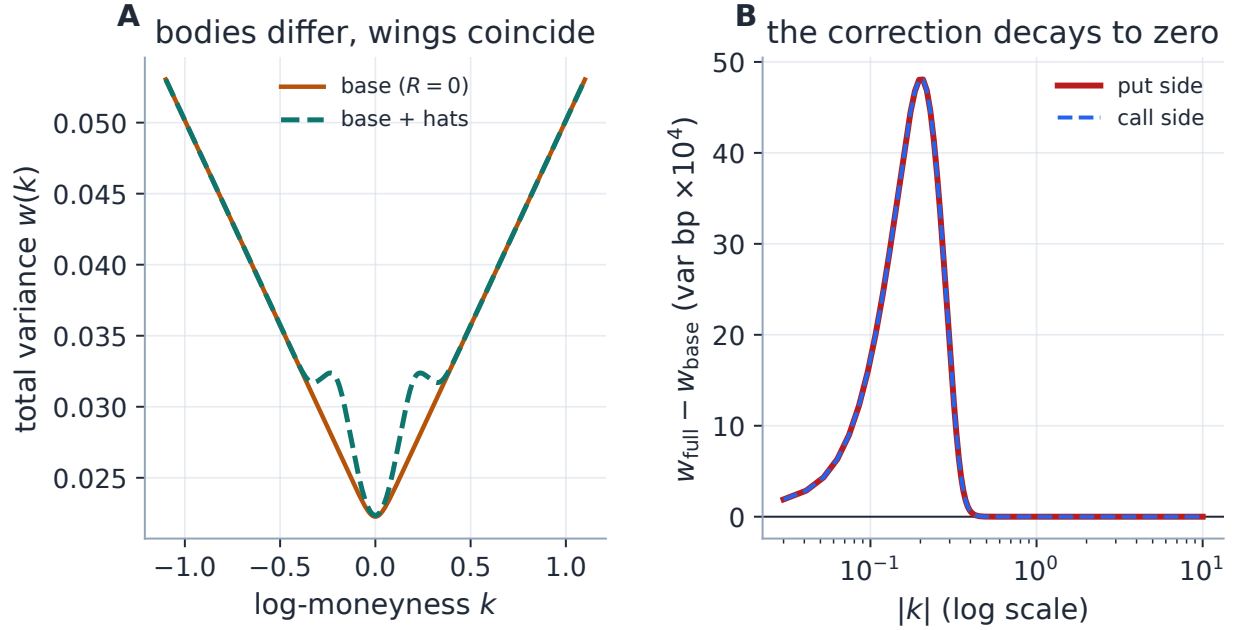


Figure 4: The correction is local. Panel A: base and base-plus-hats differ in the body but their total-variance wings coincide exactly. Panel B: the difference $w_{\text{full}} - w_{\text{base}}$ is a bump that decays to zero in both tails (the put and call sides overlie), so the wing slopes are preserved pointwise, not merely on average.

There is one honest subtlety, and it is worth stating in Note 02’s vocabulary. *Wing-neutral* is weaker than *Lee-compatible*. The hats preserve the base wing slopes, but nothing constrains those base slopes to be admissible in the first place. In total-variance k -space the wing slopes are

$$\beta_P = \frac{\sqrt{\tau}}{\sigma_{\text{ref}}} \left(\frac{2K_0}{\kappa_P} - S_0 \right), \quad \beta_C = \frac{\sqrt{\tau}}{\sigma_{\text{ref}}} \left(S_0 + \frac{2K_0}{\kappa_C} \right), \quad (8)$$

and admissibility asks $0 \leq \beta_P, \beta_C \leq 2$ (upward wings under Lee’s cap). The calibration bounds of section 6 do *not* enforce this: the hats guarantee the base slopes are *preserved*, and whether the preserved slopes are themselves Lee-clean is a separate, diagnosed condition (the cross-model wing treatment is Note 09).

The cores count R is the user’s “cores” slider — the direct analogue of the LQD Legendre order, the one dial that trades expressiveness for parsimony. The rest of the note is about reading that dial correctly.

5 Static no-arbitrage

Butterfly-freedom is the Durrleman condition, evaluated by converting the z -space variance and its derivatives to total-variance k -space via $k = \sigma_{\text{ref}} \sqrt{t} z$ and $w = tv$:

$$g(k) = \left(1 - \frac{k w'}{2w} \right)^2 - \frac{w'^2}{4} \left(\frac{1}{w} + \frac{1}{4} \right) + \frac{w''}{2} \geq 0. \quad (9)$$

Unlike LQD, and unlike the base alone, *MCS is not arbitrage-free by construction*. A signed hat can drive the raw variance negative (there is no positivity penalty in the fit) or, more commonly, break convexity locally, so the equivalence “ $g \geq 0 \iff$ non-negative density” comes with conditions worth stating: it presumes $v_R > 0$, a smooth slice, and appropriate wings, and on a finite grid g is a *diagnostic*, never a global certificate.

Remark 1 (Which curve’s g ?). Positivity and the floor interact in code. Pricing floors the variance at 10^{-8} , and the reported diagnostic is the Durrleman functional of that *priced* (floored) curve: where the floor binds, the priced slice is locally constant, its derivatives are zero, and the separate raw-positivity check flags the slice as not butterfly-free. (An earlier revision mixed the floored *value* with the raw *derivatives* — a functional of no curve at all; fixed and test-locked.) The *penalty*-path g inside the calibrator deliberately keeps the raw derivatives instead: at a collapsed-variance grid point the $1/w$ term then explodes negative, a harsh de-facto barrier against hats that drive variance through zero.

5.1 Where the arbitrage lives, and the put-wing regularizer

When cores over-reach they break (9) not near the money — where dense quotes discipline the curvature — but out in the sparsely quoted wings, where each hat’s local curvature is unconstrained by data. The offline backtest localizes the violation sharply: across the audited over-parametrized spike-regime refits, about 64% of the $g < 0$ points sit in the *put* wing (the steep, variance-loaded side of the equity skew) against only $\sim 4\%$ near the money, with the worst point at a median standardized moneyness $z \approx -3.2$. To discipline that region the joint refine carries an optional *put-wing Durrleman penalty*: on a grid $\{z_j\}$ of $M = 49$ points spanning the quoted range extended $\Delta z = 2$ into both tails, it appends the rows

$$r_j^{\text{wing}} = \sqrt{\lambda_j} \max(-g(z_j), 0), \quad \lambda_j = \Lambda \lambda_0 (1 + \mathbf{1}\{z_j < 0\}), \quad (10)$$

with base strength $\lambda_0 = 1000$, surfaced dial $\Lambda = \text{sivWingPenaltyPct}/100$ (product default 1; 0 disables), and the indicator *doubling* the weight on the put side to match the measured asymmetry. Because $\max(-g, 0) = 0$ wherever the slice is butterfly-free, (10) is exactly zero on an admissible fit: a liquid, arbitrage-free name is byte-identical with or without it. The penalty bites only where a hat would otherwise manufacture a wing violation.

Figure 5 demonstrates the penalty on a genuine over-reach. A convex truth is handed two arbitrated put-wing quotes (a localized non-convex kink of the kind a per-strike de-Americanization can produce, Note 05); a hat chases the kink and drives g to -2.45 in the put wing. The same quotes refit with the penalty on are pulled back to the admissible boundary ($g \geq -0.03$) — at the cost of fitting the arbitrated quotes less, which is the right trade.

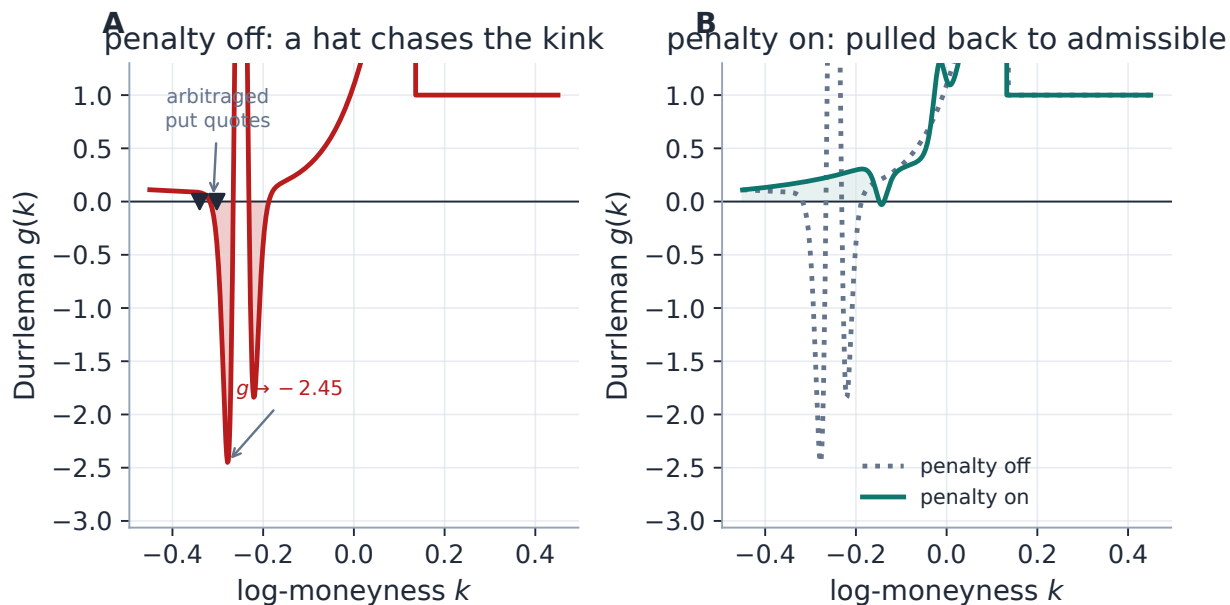


Figure 5: The put-wing penalty. Panel A: with the penalty off, a hat chases two arbitrated put quotes and breaks convexity, $g \rightarrow -2.45$ in the put wing (the positive spike is the local hump the kink induces; the y -axis is clipped to the violation). Panel B: with the penalty on, the fit is pulled back to $g \geq -0.03$ there, fitting the bad quotes less. The hinge is one-sided and zero wherever $g \geq 0$.

Heuristic

The penalty is a one-sided hinge on the same g the model already reports. Although its grid covers the quoted region too, it *bites* only where the model extrapolates: where quotes are dense they discipline the curvature, so $g \geq 0$ there and the rows are zero. It does not *guarantee* $g \geq 0$ (a soft penalty trades the violation against the data fit), but on genuinely arbitrated illiquid wings it shrinks the worst violation by orders of magnitude — the backtest measured a median wing min g moving from ≈ -7.9 to ≈ -0.02 .

Beyond the core diagnostic and this penalty, two further layers stand at increasing distance from the fit: an always-on advisory that measures the remaining extrapolated-region $g < 0$ per fit, and an export-only wing projection that raises exported prices to convexity while leaving the fitted core pinned (Note 09). None of them turns a fitted MCS slice into a certified butterfly-free object; together they bound, measure, and clean at the boundary.

6 Calibration

The fit is a three-stage workflow that respects the model's layered structure.

1. **Base fit** ($R = 0$). Fit the six base parameters to the mid quotes under bound constraints, from a data-driven start that reads the ATM variance, the local skew and the local convexity off three interpolated points. This stage is always mid, so its residual is meaningful for the next.
2. **Greedy hat seeding**. Compute the base residual in variance space and place each hat at the largest remaining $|\text{residual}|$, masking a neighbourhood after each placement so two hats cannot

seed on one feature. Each seed starts at half-width $h = 0.40$ and steepness $\kappa = 5.0$, its amplitude read from the local residual and clipped to $[-1, 1]$.

3. **Joint bounded refine.** Refine all $6+4R$ parameters together under the requested objective (mid or band) with a mild ridge $\sqrt{\varrho} \alpha_r$ on the hat amplitudes, $\varrho = 0.01$. The ridge keeps overlapping cores from exploding against each other without biasing a well-determined amplitude.

All positive parameters (K_0 , the steepnesses, the hat half-widths and steepnesses) are bound-constrained directly through scipy’s trust-region reflective solver, the hat half-width held in $[0.15, 1.5]$ and its steepness in $[1.0, 12.0]$.

The identifiability cap. The cores count is capped by the quote count,

$$R \leq \left\lfloor \frac{N_{\text{quotes}} - 6}{4} \right\rfloor, \quad (11)$$

so the model never has more free parameters than quotes, guarding sparse short-dated chains against spurious narrow kernels. Be precise about what this does and does not prevent: it caps only the *hats*, so with $N_{\text{quotes}} \geq 6$ it stops under-identified hats — but the application’s minimum of five included quotes per node means the six-parameter base can still be fitted to five quotes (one parameter over the data; the bounds and the data-driven start keep it tame, but it is not a guarantee).

The hard cap, and non-unique hats. Independently of (11), the surfaced slider is *hard-capped at two*, $R \in \{0, 1, 2\}$: section 7 shows a third core buys almost nothing out-of-sample while manufacturing wing arbitrage, so any incoming setting above two is silently clamped. Precision matters: the clamp lives in the *schema*, not the calibrator — the library `calibrate_sigmoid(n_cores=3)` runs happily and its tests exercise it. The cap is a product decision at the settings boundary; research code below it keeps the full family. Relatedly, the hats are *non-unique* by construction: two overlapping hats can trade amplitude against each other, so the round-trip test recovers the *curve*, not the parameters, and nothing downstream ever consumes a hat parameter. The individual $(\alpha_r, c_r, h_r, \kappa_r)$ are not trader handles.

The refine stage carries the same optional blocks as the other models — band fit (Note 07), var-swap target (Note 08), the model-agnostic calendar hinge (Note 10), prior persistence (Note 13), the put-wing regularizer (10), and the default-off tapered extrapolated-region fences (Notes 09–10) — each byte-identical to its own absence, so the sigmoid overlay is no longer a prior exception.

Performance

The dominant cost of an R -core fit is the $(6 + 4R + 1)$ -evaluation finite-difference Jacobian. The closed-form Jacobian of the residual stack (data + ridge + calendar), propagated through the log-cosh primitive, replaces it in one pass: a fresh run measured $161.3 \text{ ms} \rightarrow 68.3 \text{ ms}$ ($2.36\times$), optimum unchanged to 1.5×10^{-15} in cost. Scope of that number — it is the $R = 2$ mid *final-refine* microbenchmark alone (base fit, seeding and the product-default wing penalty excluded), single-threaded, not an end-to-end calibration timing. The analytic path is gated to the var-swap/prior-free configuration. When the wing penalty (10) or the extrapolation fences are active the Jacobian is *hybrid*: analytic for the data/ridge/calendar rows and finite-difference only for the small penalty blocks, so the penalized fit keeps most of the analytic speed. See appendix B.

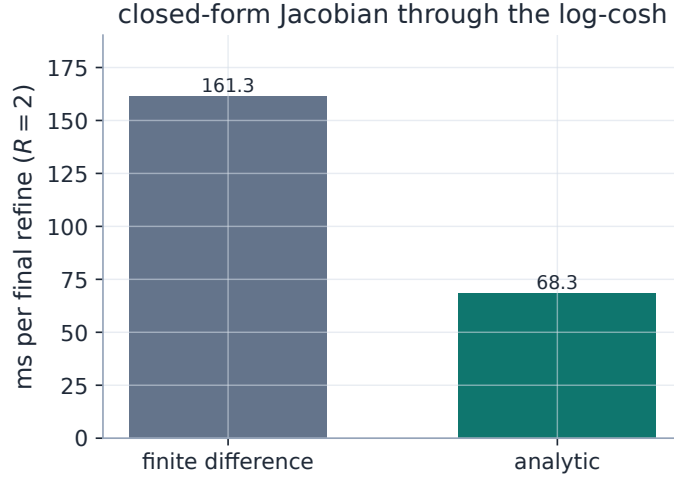


Figure 6: The closed-form Jacobian through the log-cosh replaces the $(6+4R+1)$ -evaluation finite difference on the $R = 2$ final refine ($2.36\times$ here; machine-dependent, protocol in appendix B).

Exercise 2. The centre curvature is $B''(c) = (\text{sech}^2(\kappa h/2) - 1)/\Phi_\kappa(h)$. Show it is strictly negative for every $h, \kappa > 0$, and deduce that a positive amplitude always produces a local hump and a negative amplitude a local notch — so the *sign* of α_r is the only thing a reader needs to know a core’s qualitative effect.

7 Expressiveness on a budget

A flexible model has two failure modes, and MCS shows both cleanly. Too few cores and it *underfits*: on the WW target one convex base leaves a 153.0 vol bp miss, and — because a single off-centre hat cannot address two symmetric shoulders — one core barely helps (154.7 bp); only at $R = 2$ does the error collapse, to 2.5 bp (fig. 7A). Too many cores and it *over-fits*: on a liquid, genuinely convex smile fitted to noisy quotes, each extra core drives the in-sample error down but the error against the *true* curve up — from 23.9 vol bp at $R = 0$ to 37.7 bp at $R = 3$, the classic bias–variance scissor (fig. 7B). The cores are chasing quote noise.

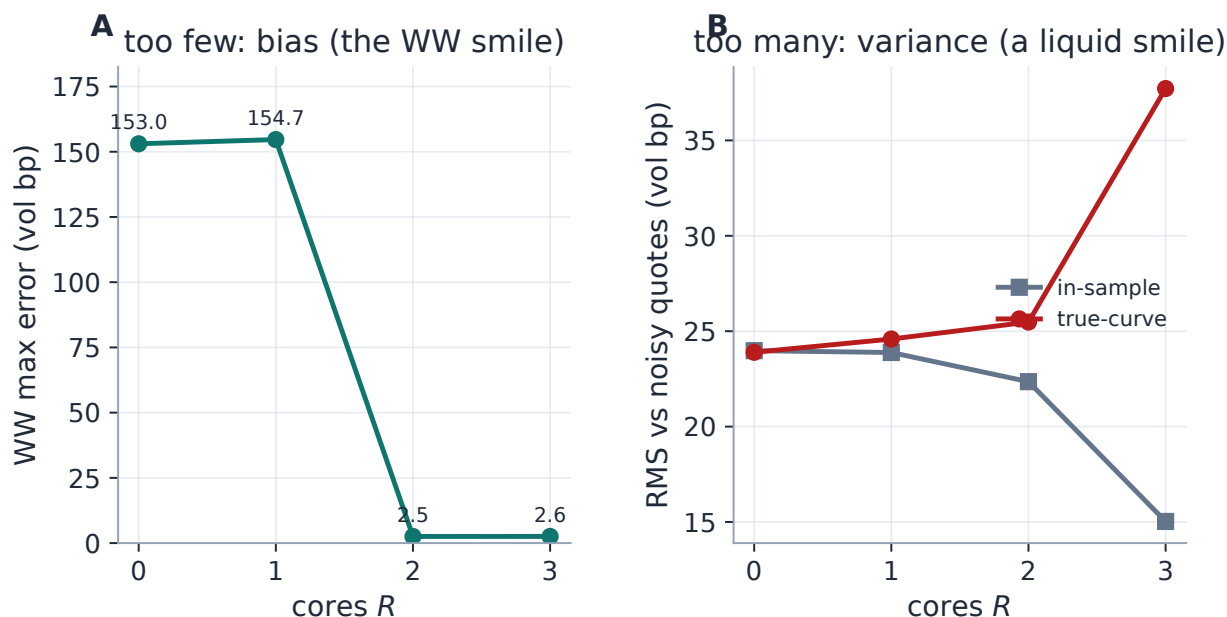


Figure 7: The two failure modes of the wrong core count. Panel A (a WW smile): too few cores is bias — the error stays near 153.0 vol bp until two hats can seat both shoulders, then collapses; a third buys nothing. Panel B (a noisy liquid smile): too many cores is variance — the in-sample error keeps falling while the true-curve error rises, the signature of over-fitting. The right core count is neither the most nor the fewest.

This is why the slider is governed rather than free. The backtest made the cost concrete on real data: on liquid index chains the extra cores chase quote noise and manufacture butterfly arbitrage (the analytic diagnostic flags $g < 0$ on a majority of nodes at $R \geq 2$ on dense data), while the base $R = 0$ behaves comparably to SVI. The honest marginal number for the third core, from the spike replay: out-of-sample RMS 13.99→13.58 bp — a 0.41 bp gain — for roughly four times the fit cost (514 → 2023 ms in that harness) and worse arbitrage. Not negative precision, but nowhere near worth it. Hence the hard cap at two, on top of the identifiability cap, the amplitude ridge, and the g diagnostic.

Caution

With great expressiveness comes over-fitting, and MCS is the one model in the series where the default posture is restraint. The operational guidance: *use cores only when the smile is genuinely non-convex* (events, bimodal positioning), and prefer LQD or Local-Vol for routine fitting. The cores slider is a scalpel, not a default — which is why the product default model is LQD, and MCS is the event/WW specialist.

Case file: two defences, one pathology

The put-wing violation has *two* sources: the flexible model genuinely over-reaching (cured by the penalty (10)), and *arbitraged inputs* — the per-strike de-Americanization of Note 05 can hand every model a call curve already slightly non-convex in the wings. An ablation isolating the two defences (the de-Am convex repair of Note 05 versus the penalty) on illiquid American

ETFs found them *complementary, not redundant*. On the arb-prone population (medians over 38 nodes): with neither, worst $g \approx -30$ and in-sample RMS 92 bp; the de-Am repair alone cuts the violation about threefold *and lowers* the RMS to 25 bp (it removes the arbitrated quotes the model was chasing); the penalty alone nearly eliminates the violation but at 749 bp (it must fight those same quotes); both attain the penalty's arbitrage reduction at 225 bp, because the inputs are cleaned before the constraint is imposed. Reduction, not elimination: even with both defences on, 26% of the population stays flagged. One node makes it vivid (EFA in the Aug-2024 spike, 11 days, 22 quotes): neither, 75 bp with worst $g = -116$; repair alone, a tight 18 bp but $g = -12$; penalty alone, arbitrage-free but 726 bp; both, arbitrage-free at 34 bp. *The fence is expensive only when it must fight arbitrated inputs — cleaning the inputs at the source is what makes the constraint affordable.*

8 Worked example: a WW smile

The target in fig. 1 is a synthetic WW built to be *globally* admissible, not merely clean on a window. Its total variance is a hyperbolic base plus two Gaussian shoulders, $w(k) = a + b\sqrt{k^2 + \varsigma^2} + \sum_{\pm} A e^{-((k \mp c)/s)^2}$. Gaussians vanish with all derivatives, so the w -wings are exactly linear with slope $\beta = 0.055 \leq 2$ (Lee-admissible for every k), and g tends to the positive limit $(4 - \beta^2)/16 = 0.250$ in both tails. The generator computes g from analytic jets (no finite differences, the measurement lesson of Note 02) and asserts $g > 0$ for target *and* fit on a dense grid to $|k| = 12$ before writing anything.

The base ($R = 0$) fits it only to 153.05 vol bp, missing both shoulders. Two hats ($R = 2$, $6 + 4 \cdot 2 = 14$ parameters) reduce the maximum error to 2.55 vol bp — report two numbers and keep them separate: the *quote* error 2.55 vol bp is the fit's miss on the target, while the fit stays globally butterfly-clean, its wide-grid minimum $g = 0.28$ (the target's is 0.25). Meanwhile the preserved base wing slopes $(S_0 - 2K_0/\kappa_P, S_0 + 2K_0/\kappa_C) = (-0.017, 0.017)$ in z -space are exactly the base's, cores or no cores. Figure 8 plots the diagnostic for both curves.

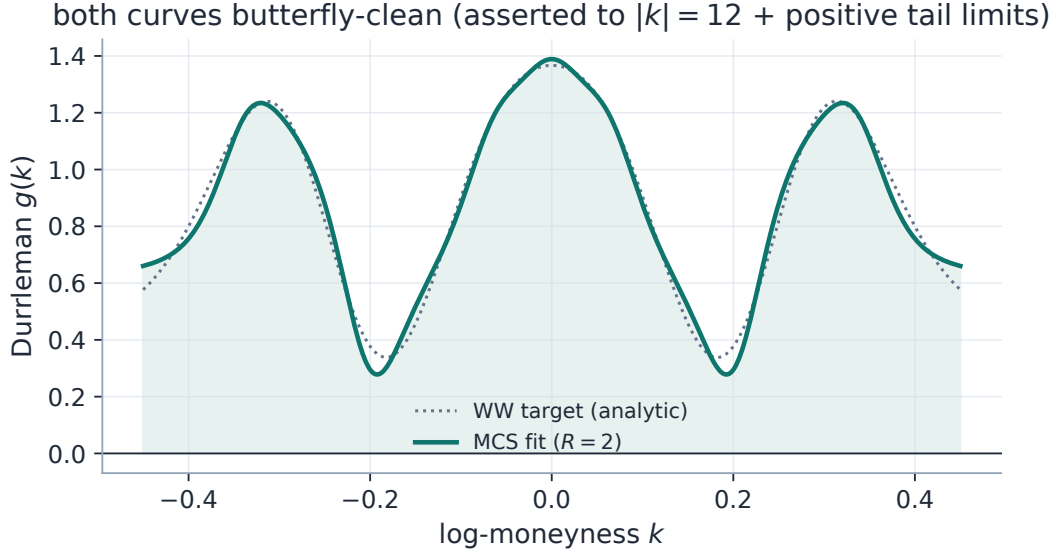


Figure 8: The butterfly diagnostic for the fitted slice and the target it chased. Both stay non-negative on the display window; with the wide-grid assertion to $|k| = 12$ and the positive analytic tail limits, the cleanliness claim is global, not per-window.

Remark 2 (The target must be clean too — a bug fixed twice). A diagnostic is only as strong as the exercise it certifies, and this figure’s target needed two rounds of honesty. An early revision built a WW target whose *own* Durrleman function dipped to $g \approx -0.38$ near the shoulders: the synthetic “market” itself carried butterfly arbitrage, so no admissible slice could both track it and honour $g \geq 0$, and the figure’s claim was quietly false. The first fix re-chose the shoulder amplitudes and widths (the shoulder-top curvature scales like $2A/s^2$, which drives w'' , hence g , negative) — but that target was built in *volatility* space with linear tails, so its total variance grew quadratically and still violated Lee far outside the window: cleanliness only per-window. The current construction builds the target in *total-variance* space, so the w -wings are exactly linear and the tail g limit is positive analytically. The lesson is the same one Note 02 drew about measuring g : compute it from analytic jets, and certify globally (a wide grid plus a tail limit), never on a plotted window alone.

Exercise 3. The identifiability cap (11) counts parameters against quotes. Show that with $N_{\text{quotes}} = 5$ it gives $R = 0$, yet the base alone has six parameters — one more than the data. Explain why this is not an outright failure (bounds and the data-driven start regularize it) but is also not an identifiability guarantee, and why a desk should distrust the individual base parameters, though not necessarily the fitted curve, on such a chain.

9 What is original, and limitations

The genuinely original elements are the *zero-wing hat* ((6)–lemma 1) — a correction basis local in the body and provably neutral in the wings, so expressiveness and tail control are decoupled by an elementary annihilation property — and the *layered three-stage calibration* that places the corrections on residuals rather than optimizing them blind. Together they let one model span from a clean index smile ($R = 0$, a convex family in its own right, not an SVI equivalent) to an

event-shaped WW smile ($R = 2$) on a single slider. (A WW *implied-volatility* curve suggests, but does not by itself prove, a bimodal risk-neutral density: the density is the Breeden–Litzenberger second derivative of price, not a re-reading of the vol curve.)

Limitations, stated plainly. MCS is *not* arbitrage-free by construction: signed hats can drive variance negative or break convexity, so g is a reported diagnostic and the wing penalty a soft, non-guaranteeing fence (section 5). Its wings are wing-*neutral*, not Lee-guaranteed: admissibility of the preserved base slopes is a separate check (8). It over-fits liquid data when the slider is pushed, which is why the surfaced cap is two and the default model is LQD. And its hats are non-unique, so comparisons across fits must be made on curves and handles, never on the correction parameters. The cores slider earns its keep exactly when the smile is genuinely non-convex, and nowhere else.

A Hyperparameter atlas

Table 2: Multi-Core Sigmoid (MCS) hyperparameters.

Knob	Default	Role
<i>Surfaced. Product defaults shown (the product’s default model is LQD; these apply when MCS is selected). Library defaults differ: $R = 0$, penalty off.</i>		
nCores R	2	Number of zero-wing hats, $R \in \{0, 1, 2\}$ (schema-clamped at two on every parse; the library is uncapped, and (11) may reduce R further). 0 = pure base.
sivWingPenaltyPct Λ	100	Put-wing Durrleman penalty (10) strength; 100 = base $\lambda_0 = 1000$, 0 = off.
sigmoidRidge ϱ	0.01	ℓ^2 penalty on hat amplitudes in the refine stage.
midAnchorWeight	0.05	Band-fit mid anchor (Note 07).
weightScheme	equal	Per-quote weights (Note 07).
calendarWeight	10^6	Calendar hinge penalty (Note 10).
extrapEnforce	off	Tapered extrapolated-region fences (Notes 09–10); hybrid Jacobian when on.
<i>Internal (seeding / bounds)</i>		
_H_INIT	0.40	Hat half-width seed.
_KAPPA_INIT	5.0	Hat steepness seed.
_H_BOUNDS	[0.15, 1.5]	Hat half-width bounds.
_KAPPA_BOUNDS	[1.0, 12.0]	Hat steepness bounds.
_C_PAD	0.5	Centre padding beyond the quoted z -range.
_V_FLOOR	10^{-8}	Variance floor so $\sqrt{v}$ stays real.
_LOGCOSH_CLIP	50	$ x $ beyond which log cosh uses its asymptote.
xtol/ftol	10^{-12}	trf tolerances.

B Performance notes

1. **Analytic Jacobian** through the log-cosh primitive replaces the $(6 + 4R + 1)$ -evaluation finite difference; measured $2.36\times$ on the $R = 2$ mid *final-refine* microbenchmark (base fit, seeding and the product-default wing penalty excluded; single-threaded development machine, fastest of several), optimum unchanged to 1.5×10^{-15} . Machine-dependent numbers belong here, not in the body. Gated to the var-swap/prior-free path; hybrid (analytic core + FD penalty blocks) under the wing penalty or extrapolation fences.
2. **Layered warm start.** Fitting the base first and seeding hats on its residual gives the joint refine an excellent start, so the bounded trf converges quickly even with several cores.
3. **Vectorized kernels.** Φ, Φ', Φ'' and the hat second differences are NumPy-vectorized and side-effect free, shared between the model and its Jacobian — one set of jets, computed once.
4. **Identifiability cap.** Limiting R by the quote count avoids wasting iterations on under-determined narrow kernels, and is the first line of defence against the over-fit of section 7.

C Traceability

A scoping word first. The “zero-wing / diagnostics” rows anchor this note’s current construction; the legacy over-parametrized golden tests lock the library’s uncapped behaviour and are kept as regression tripwires, not as evidence for these figures — the regenerated two-core example has its own lock. The penalty rows are locked with stated tolerances: penalty-off is byte-identical; penalty-on over a clean slice is unchanged to 10^{-9} relative; and the repair test accepts a repaired $\min g \geq -0.05$, not exactly zero. A finite diagnostic grid is not a global proof.

Table 3: Claims in this note and the code/tests that lock them.

Claim	Object	Code anchor <i>Test anchors</i>
Hat is unit-height, flat at centre, zero-wing in value/slope/curvature	lemma 1	<code>models/sigmoid/kernels.py</code> <code>test_sigmoid.py::test_hat_is_zero_wing_and_unit_height</code>
Wing slopes preserved under cores; g and $\min v$ diagnostics match note	(5)	<code>models/sigmoid/sigmoid.py</code> <code>test_sigmoid.py::test_note_arbitrage_diagnostics</code>
Reported g is the priced (floored) curve’s functional; degenerate slices flagged not-free	section 5	<code>models/sigmoid/sigmoid.py::gatheral_g</code> <code>test_sigmoid.py::test_floored_diagnostic_is_priced_curve_functional</code>
This note’s two-core WW example (fit tight, base misses shoulders; target and fit globally butterfly-clean)	section 8	<code>models/sigmoid/calibrate.py</code> <code>test_sigmoid.py::test_note03_ww_two_core_example_regression</code> <code>::test_note03_ww_target_globally_clean</code>

Claim	Object	Code anchor <i>Test anchors</i>
Cores improve the WW fit monotonically (legacy synthetic)	section 7	models/sigmoid/calibrate.py test_sigmoid.py::test_more_cores_fit_better_on_ww_smile
Hat-count cap $R \leq \lfloor (N - 6)/4 \rfloor$ (hats only; the base still fits at $N = 5$)	(11)	models/sigmoid/calibrate.py test_sigmoid.py::test_cores_are_capped_to_quote_count
Surfaced slider clamped to two (schema boundary, every parse)	section 6	api/schemas.py test_siv_wing_penalty.py::test_cores_clamped_to_two
Put-wing hinge repairs wing arb (to $\min g \geq -0.05$); clean slices unchanged (10^{-9} rel); off by library default (byte-identical)	(10)	models/sigmoid/calibrate.py test_siv_wing_penalty.py::test_penalty_removes_wing_arb ::test_penalty_byte_identical_on_arb_free_slice ::test_penalty_off_default
Analytic Jacobian matches FD (base, cores, band, calendar)	section 6	models/sigmoid/jacobian.py test_sigmoid_jacobian.py::test_two_cores_mid ::test_two_cores_band ::test_calendar_active
Curve identified, parameters deliberately not	section 6	models/sigmoid/calibrate.py test_sigmoid.py::test_round_trip_curve_recovery
Prior blocks reach the MCS overlay	section 6	models/sigmoid/calibrate.py test_prior_parametric.py::test_operator_prior_pulls_all_models_toward_prior_skew
Over-parametrized ablation numbers (two defences, one pathology)	section 7	backend/backtest/ablation_arb.py test_ablation_arb.py

D Reference implementation

The full slice $v_R(z)$ (7) and the butterfly diagnostic $g(k)$ (9) are a few lines built on the production kernels. The z -jets (value, slope, curvature) are carried together because g needs all three; the zero-wing second difference is written once and reused for the hat and its derivatives. The listing is imported and executed by `figures/gen_mcs_corrections.py`, which asserts it agrees with the production `MultiCoreSiv` to 10^{-13} before drawing a figure, so the code printed here is the code that produced the note's numbers.

Listing 1: Executable reference maps used by this note (`figures/mcs_corrections_reference.py`).

```
1 """Executable reference maps for the "base and correction" edition of
   Note 03.
```

```

2
3  Two functions are the note's Appendix D listing.  'v_model'
   evaluates the
4  Multi-Core Sigmoid variance by *superposition* --- a convex base
   carries the
5  tails, and each signed zero-wing hat adds a local body correction ---
   returning
6  the three z-jets (value, slope, curvature) together because the
   butterfly
7  diagnostic needs all three.  'durrleman_g' converts those jets to
   total-variance k-space and evaluates the density factor. Both are
8  built from
9  the *production* kernel primitives, and the generator asserts they
   agree with
10 the production 'MultiCoreSiv' to 1e-13 before drawing a single
   figure, so the
11 listing printed in the note is the code that produced its numbers.
12 """
13
14 from __future__ import annotations
15
16 import numpy as np
17
18 from volfit.models.sigmoid.kernels import (
19     hat, hat_p, hat_pp, phi, phi_p, phi_pp,
20 )
21
22
23 def v_model(z, base, cores):
24     """MCS variance  $v_R(z) = v_{base}(z) + \sum_r \alpha_r B_r(z)$ , with
       its z-jets.
25
26     'base' is (v0, s0, k0, z0, kappa_p, kappa_c); 'cores' is a
       list of
27     (alpha, c, h, kappa). The base is one convex log-cosh slice that
       owns the
28     asymptotic wings; every hat is a centred second difference that
       vanishes ---
29     in value, slope, and curvature --- in both tails, so the
       corrections reshape
30     the body without moving the wing slopes.
31     """
32     v0, s0, k0, z0, kp, kc = base
33     u = np.asarray(z, dtype=float) - z0
34     kappa = np.where(u < 0.0, kp, kc) # asymmetric
       wings,  $C^2$  at z0
35     v = v0 + s0 * u + k0 * phi(u, kappa)
36     vz = s0 + k0 * phi_p(u, kappa)
37     vzz = k0 * phi_pp(u, kappa)
38     for alpha, c, h, kap in cores: # add the signed
       zero-wing hats

```

```

39         v = v + alpha * hat(z, c, h, kap)
40         vz = vz + alpha * hat_p(z, c, h, kap)
41         vzz = vzz + alpha * hat_pp(z, c, h, kap)
42     return v, vz, vzz
43
44
45 def durrleman_g(z, base, cores, t, sigma_ref, v_floor=1e-8):
46     """Butterfly diagnostic  $g(k) \geq 0$ , converting the  $z$ -jets to  $k$ -
47         space.
48
49      $k = \sigma_{\text{ref}} \sqrt{t}$   $z$  and  $w = t v$ , so  $w'$  and  $w''$  pick up
50          $\sigma_{\text{ref}}/\sqrt{t}$ 
51         factors.  $g$  is proportional to the risk-neutral density, so  $g \geq 0$ 
52         on the
53         grid is the no-butterfly condition; it is a diagnostic on a finite
54         grid, not
55         a global certificate.
56     """
57     v, vz, vzz = v_model(z, base, cores)
58     v = np.maximum(v, v_floor)
59     k = sigma_ref * np.sqrt(t) * np.asarray(z, dtype=float)
60     w = t * v
61     wk = np.sqrt(t) / sigma_ref * vz
62     wkk = vzz / sigma_ref**2
63     return (1.0 - k * wk / (2.0 * w)) ** 2 - 0.25 * wk**2 * (1.0 / w +
64         0.25) + 0.5 * wkk

```

References

- [1] J. Gatheral. *The Volatility Surface: A Practitioner's Guide*. Wiley, 2006.
- [2] V. Durrleman. From implied to spot volatilities. *Finance and Stochastics*, 14(2):157–177, 2010.
- [3] R. W. Lee. The moment formula for implied volatility at extreme strikes. *Mathematical Finance*, 14(3):469–480, 2004.